# Git and GitHub

A step-by-step guide for complete beginners

MERN Workshop

> **How to use this guide**
> Every command in this guide comes with three things: **what it does**, **why you run it**, and **what you will see** on your screen. You do not need to memorise anything. Follow the steps in order, and come back here whenever you forget a command.

# 1. What are Git and GitHub?

**Git** is a program on your computer that remembers every version of your files. Think of it like the *save history* in a video game: you can save at any point, and go back to any earlier save if something breaks.

**GitHub** is a website where you store your Git projects online. It is like Google Drive, but made for code. It also lets teachers and students share work.

| | Git | GitHub |
|---|---|---|
| What is it? | A program on your laptop | A website (github.com) |
| Needs internet? | No | Yes |
| Job | Tracks changes to your files | Stores your project online and shares it |
| Example | `git commit` | Clicking the Fork button |

### Why do we use them in this workshop?

- You get all the tasks and examples from the teacher with one command.
- Your work is saved online, so you never lose it if your laptop breaks.
- The teacher can see your progress and help you.
- Every software company uses Git, so you are learning a real job skill.

# 2. Words you will hear (glossary)

| Word | Simple meaning |
|---|---|
| Repository (repo) | A project folder that Git is watching. Ours is called **mern-workshop**. |
| Fork | Your own copy of the teacher's repo on GitHub. You can change it freely without breaking the original. |
| Clone | Download a repo from GitHub onto your laptop. |
| Local | On your laptop. |
| Remote | On GitHub (online). |
| origin | The nickname Git gives to **your fork** on GitHub. |
| upstream | The nickname we give to **the teacher's original repo**. |
| Stage | Pick which changed files you want to include in your next save. |
| Commit | A save point. A snapshot of your files with a short message. |
| Push | Upload your commits from your laptop to GitHub. |
| Pull / Fetch | Download new changes from GitHub to your laptop. |
| Branch | A separate line of work. We only use **main**, the default one. |
| Merge | Combine changes from two places into one. |

# 3. The big picture

There are three places your files live. Understanding this makes every command make sense.

```
TEACHER'S REPO (GitHub)          <- "upstream"  (you only READ from it)
         |
         |  Fork (click the button once)
         v
YOUR FORK (GitHub)               <- "origin"    (you PUSH your work here)
         ^   |
   push  |   |  clone (once)
         |   v
YOUR LAPTOP (local folder)       <- where you write code
```

Your daily routine is always the same:

- **Get** new tasks from the teacher (upstream) to your laptop.
- **Work** on a task in VS Code.
- **Save** your work with `add` and `commit`.
- **Upload** it to your fork with `push`.

# 4. Install and check Git

Open a terminal first. On **Mac**: press Cmd + Space, type *Terminal*, press Enter. On **Windows**: press the Windows key, type *PowerShell*, press Enter.

```
git --version
```

| | |
|---|---|
| **WHAT IT DOES** | Prints the version of Git installed on your computer. |
| **WHY YOU RUN IT** | To find out if Git is already installed. If it prints a version, you are ready. If you see an error, you need to install Git (next). |
| **YOU WILL SEE** | `git version 2.45.0` |

## If Git is not installed

**Mac**

```
xcode-select --install
```

| | |
|---|---|
| **WHAT IT DOES** | Installs Apple's developer tools, which include Git. |
| **WHY YOU RUN IT** | Mac does not always come with Git. This is the easiest way to get it. A popup appears: click **Install** and wait a few minutes. |

**Windows**

```
winget install --id Git.Git -e --source winget
```

| | |
|---|---|
| **WHAT IT DOES** | Downloads and installs Git using Windows' built-in app installer. |
| **WHY YOU RUN IT** | Windows does not come with Git. Alternatively, download the installer from git-scm.com/download/win and click Next on every screen. |

> **Important**
> After installing, **close the terminal and open a new one**. Otherwise it will not find Git yet. Then run `git --version` again.

# 5. Tell Git who you are (one time only)

Every save point (commit) has a name and email on it, so people know who made it. You only do this once per computer. Use the **same email as your GitHub account**.

```
git config --global user.name "Your Name"
```

| | |
|---|---|
| **WHAT IT DOES** | Stores your name in Git's settings. `--global` means 'for every project on this computer'. |
| **WHY YOU RUN IT** | Without it, Git refuses to save and shows 'Author identity unknown'. Keep the quotation marks. Replace Your Name with your real name. |

```
git config --global user.email "you@example.com"
```

| | |
|---|---|
| **WHAT IT DOES** | Stores your email in Git's settings. |
| **WHY YOU RUN IT** | It links your commits to your GitHub account so they show up under your profile. |

```
git config --global init.defaultBranch main
```

| | |
|---|---|
| **WHAT IT DOES** | Sets the default branch name to **main**. |
| **WHY YOU RUN IT** | GitHub uses **main**. This keeps your laptop and GitHub using the same name. |

Line endings setting (just copy the one for your computer):

**Mac**

```
git config --global core.autocrlf input
```

**Windows**

```
git config --global core.autocrlf true
```

Why? Windows and Mac mark the end of a line differently in text files. This setting stops Git from showing a whole file as 'changed' when you did not change it.

```
git config --global --list
```

| | |
|---|---|
| **WHAT IT DOES** | Shows all the settings you saved. |
| **WHY YOU RUN IT** | To double-check that your name and email are correct. |
| **YOU WILL SEE** | `user.name=Your Name`<br>`user.email=you@example.com`<br>`init.defaultbranch=main` |

# 6. Sign in to GitHub from your laptop

To upload (push) your work, GitHub needs to know it is really you. **Your account password will NOT work.** The easiest way is GitHub CLI, a small helper program.

**Mac**

```
brew install gh
```

**Windows** (close and reopen the terminal afterwards)

```
winget install --id GitHub.cli -e --source winget

gh auth login
```

| | |
|---|---|
| **WHAT IT DOES** | Starts a guided login. It asks questions in the terminal. |
| **WHY YOU RUN IT** | It connects your laptop to your GitHub account once, so every later push just works. |
| **YOU WILL SEE** | `? Where do you use GitHub?  GitHub.com`<br>`? Preferred protocol?  HTTPS`<br>`? Authenticate Git with your GitHub credentials?  Yes`<br>`? How would you like to authenticate?  Login with a web browser` |

Use the arrow keys and Enter to choose. It shows a one-time code. Copy it, press Enter, and the browser opens. Paste the code, click **Authorize**, and you are done.

> **No Homebrew on Mac?**
> Install it from brew.sh first, or use a Personal Access Token: GitHub > Settings > Developer settings > Personal access tokens > Tokens (classic), tick the **repo** box, and paste the token when Git asks for a password.

# 7. Fork the workshop repo

A fork is **your own copy** of the teacher's project on GitHub. This is done in the browser, not the terminal, and only once.

- Sign in at **github.com**.
- Open **https://github.com/KrunalTarale5/mern-workshop**.
- Click the **Fork** button at the top right.
- Leave the name as *mern-workshop* and click **Create fork**.
- Check the address bar. It should now say **github.com/YOUR-USERNAME/mern-workshop**.

**Why fork instead of just downloading?**
You cannot upload changes to the teacher's repo, but you can to your own fork. Forking gives you a place to save all your work online.

# 8. Clone your fork to your laptop

Cloning downloads your fork from GitHub into a folder on your computer. Replace **YOUR-USERNAME** with your GitHub username (no angle brackets).

**Mac**

```
cd ~/Desktop
```

| | |
|---|---|
| **WHAT IT DOES** | `cd` means 'change directory' (move into a folder). `~/Desktop` is your Desktop. |
| **WHY YOU RUN IT** | So the project folder gets created on your Desktop where you can find it easily. |

**Windows**

```
cd $HOME\Desktop
```

```
git clone https://github.com/YOUR-USERNAME/mern-workshop.git
```

| | |
|---|---|
| **WHAT IT DOES** | Downloads your fork (all files and their full history) into a new folder named mern-workshop. |
| **WHY YOU RUN IT** | This is how you get the tasks, examples and slides onto your laptop. You only do it once. |
| **YOU WILL SEE** | `Cloning into 'mern-workshop'...`<br>`remote: Enumerating objects: 120, done.`<br>`Receiving objects: 100% (120/120), done.` |

```
cd mern-workshop
```

| | |
|---|---|
| **WHAT IT DOES** | Moves your terminal into the project folder. |
| **WHY YOU RUN IT** | Git commands only work **inside** the project folder. Always do this first. |

```
code .
```

| | |
|---|---|
| **WHAT IT DOES** | Opens the current folder in VS Code. The dot means 'this folder'. |
| **WHY YOU RUN IT** | So you can see and edit all the files. If it says 'command not found', open VS Code manually (File > Open Folder). On Mac you can fix it: in VS Code press Cmd+Shift+P, type 'shell command', choose 'Install code command in PATH'. |

# 9. Connect to the teacher's repo (upstream)

Your fork does not update by itself. When the teacher adds new tasks, your fork will not have them. We add a second connection called **upstream** so you can pull the new material.

```
git remote add upstream https://github.com/KrunalTarale5/mern-workshop.git
```

| | |
|---|---|
| **WHAT IT DOES** | Adds a new shortcut named *upstream* that points to the teacher's repo. |
| **WHY YOU RUN IT** | So you can later download new tasks from the teacher with a short command. |

```
git remote -v
```

| | |
|---|---|
| **WHAT IT DOES** | Lists all the online connections (remotes) of this project. `-v` means 'verbose' (show details). |
| **WHY YOU RUN IT** | To confirm you have both: **origin** (your fork) and **upstream** (the teacher). |
| **YOU WILL SEE** | `origin  https://github.com/YOUR-USERNAME/mern-workshop.git (fetch)`<br>`origin  https://github.com/YOUR-USERNAME/mern-workshop.git (push)`<br>`upstream  https://github.com/KrunalTarale5/mern-workshop.git (fetch)`<br>`upstream  https://github.com/KrunalTarale5/mern-workshop.git (push)` |

> **Check**
> If the origin address has the teacher's name instead of yours, you cloned the wrong repo. Go back to step 7 and fork first.

# 10. Your daily workflow

## Part A: Get the latest tasks from the teacher

Do this at the start of every class.

```
git checkout main
```

| | |
|---|---|
| **WHAT IT DOES** | Switches to the **main** branch. |
| **WHY YOU RUN IT** | To make sure you are on the main line of work before updating. |
| **YOU WILL SEE** | `Already on 'main'` |

```
git fetch upstream
```

| | |
|---|---|
| **WHAT IT DOES** | Downloads the teacher's latest changes, but does **not** change your files yet. |
| **WHY YOU RUN IT** | It is a safe 'look' at what is new. Nothing in your folder changes. |
| **YOU WILL SEE** | `From https://github.com/KrunalTarale5/mern-workshop`<br>`* [new branch]  main  -> upstream/main` |

```
git merge upstream/main
```

| | |
|---|---|
| **WHAT IT DOES** | Combines the teacher's new files into your folder. |
| **WHY YOU RUN IT** | This actually gives you the new tasks and examples. |
| **YOU WILL SEE** | `Updating a47d33a..b81c2f0`<br>`Fast-forward`<br>` Tasks/03-express/task-12-new.md \| 20 +++++` |

```
git push origin main
```

| | |
|---|---|
| **WHAT IT DOES** | Uploads everything to your fork on GitHub. |
| **WHY YOU RUN IT** | So your online fork also has the new tasks, and stays equal to your laptop. |
| **YOU WILL SEE** | `To https://github.com/YOUR-USERNAME/mern-workshop.git`<br>`   a47d33a..b81c2f0  main -> main` |

> **Shortcut**
> On your fork page on GitHub you can click **Sync fork > Update branch**, then run `git pull` in the terminal.

## Part B: Do a task and save your work

After you finish a task (or a good part of it), save it in these 3 steps: **status, add, commit** and then **push**.

```
git status
```

| | |
|---|---|
| **WHAT IT DOES** | Shows which files you changed and whether they are saved. |
| **WHY YOU RUN IT** | Your 'what did I do?' check. Use it all the time; it never changes anything. |
| **YOU WILL SEE** | `On branch main`<br>`Changes not staged for commit:`<br>`        modified:   Tasks/01-javascript/task-01-variables.js` |

Red files are changed but not yet staged. Green files are staged and ready to commit.

```
git add .
```

| | |
|---|---|
| **WHAT IT DOES** | Stages **all** changed files. The dot means 'everything here'. To stage only one file: `git add task-01-variables.js`. |
| **WHY YOU RUN IT** | Git only saves what you have chosen. Think of putting items in a box before you seal it. |

```
git commit -m "Complete JS task 01 variables"
```

| | |
|---|---|
| **WHAT IT DOES** | Creates a save point with the message you write inside the quotes. `-m` means 'message'. |
| **WHY YOU RUN IT** | This is the actual save. The message tells you (and the teacher) what you did. Write a short, clear message such as 'Complete express task 03'. |
| **YOU WILL SEE** | `[main 4f2a9c1] Complete JS task 01 variables`<br>` 1 file changed, 12 insertions(+)` |

```
git push origin main
```

| | |
|---|---|
| **WHAT IT DOES** | Uploads your commits to **origin** (your fork), on the **main** branch. |
| **WHY YOU RUN IT** | Commit saves on your laptop only. Push puts a copy on GitHub so it is backed up and the teacher can see it. Refresh your fork page to check. |
| **YOU WILL SEE** | `Writing objects: 100% (4/4), done.`<br>`To https://github.com/YOUR-USERNAME/mern-workshop.git`<br>`   b81c2f0..4f2a9c1  main -> main` |

**The 4-command habit**
`git status` → `git add .` → `git commit -m "message"` → `git push origin main`. Do it after every task.

# 11. Looking at your history

```
git log --oneline
```

| | |
|---|---|
| **WHAT IT DOES** | Lists your save points (commits), newest first, one per line. |
| **WHY YOU RUN IT** | To see what you have saved so far. Press **q** to leave the list if it does not close by itself. |
| **YOU WILL SEE** | `4f2a9c1 Complete JS task 01 variables`<br>`b81c2f0 Add express tasks`<br>`a47d33a Initial commit` |

```
git diff
```

| | |
|---|---|
| **WHAT IT DOES** | Shows exactly which lines you changed since the last commit. Lines starting with **+** are added and **-** are removed. |
| **WHY YOU RUN IT** | To review your work before committing. Press **q** to leave. |
| **YOU WILL SEE** | `-let age = 0;`<br>`+let age = 19;` |

# 12. Fixing mistakes

Everyone makes mistakes with Git. These are the common ones and the safe fix.

```
git restore task-01-variables.js
```

| | |
|---|---|
| **WHAT IT DOES** | Throws away your changes in that file and goes back to the last saved version. |
| **WHY YOU RUN IT** | When you broke a file and want to start it again from scratch. **Warning:** the changes are gone for good, so be sure. |

```
git restore --staged task-01-variables.js
```

| | |
|---|---|
| **WHAT IT DOES** | Un-stages a file (takes it out of the box) but keeps your edits. |
| **WHY YOU RUN IT** | When you ran `git add` on a file by mistake. |

```
git commit --amend -m "Better message"
```

| | |
|---|---|
| **WHAT IT DOES** | Replaces the message of your **last** commit. |
| **WHY YOU RUN IT** | When you made a typo in the message. Only do this **before** you push. |

> **If you are stuck**
> Do not delete the folder and start over. Run `git status`, take a screenshot of what it says, and ask your teacher. Your work is almost always recoverable.

# 13. Common errors and what to do

| Error message | What it means | Fix |
|---|---|---|
| `git: command not found` 'git' is not recognized | Git is not installed, or the terminal was not reopened. | Install Git (section 4), close and reopen the terminal. |
| `Author identity unknown` | Git does not know your name/email yet. | Run the two `git config --global` commands (section 5). |
| `fatal: not a git repository` | You are in the wrong folder. | Run `cd mern-workshop`, or open the terminal from VS Code. |
| `Authentication failed` | GitHub did not accept your password. | Use `gh auth login` or a token (section 6). |
| `remote origin already exists` | You already added that connection. | Nothing to do. Run `git remote -v` to check. |
| `nothing to commit, working tree clean` | You have no new changes to save. | Fine. Edit a file first, then try again. |
| `rejected ... fetch first` (on push) | GitHub has changes you do not have. | Run `git pull origin main`, then push again. |
| `CONFLICT (content)` | You and the teacher changed the same lines. | Stop. Do not delete files. Ask your teacher for help. |

# 14. One-page cheat sheet

## One time only (setup)

| Command | Purpose |
|---|---|
| `git config --global user.name "Name"` | Set your name |
| `git config --global user.email "mail"` | Set your email |
| `gh auth login` | Sign in to GitHub |
| `git clone <your-fork-url>` | Download your fork |
| `git remote add upstream <teacher-url>` | Connect to the teacher's repo |

## Start of every class

| Command | Purpose |
|---|---|
| `cd mern-workshop` | Go into the project folder |
| `git checkout main` | Make sure you are on main |
| `git fetch upstream` | Look for new teacher changes |
| `git merge upstream/main` | Get the new tasks |
| `git push origin main` | Update your fork |

## After every task

| Command | Purpose |
|---|---|
| `git status` | See what changed |
| `git add .` | Choose all changes to save |
| `git commit -m "message"` | Save a snapshot |
| `git push origin main` | Upload to GitHub |

## When in doubt

| Command | Purpose |
|---|---|
| `git status` | What is going on right now? |
| `git log --oneline` | What have I saved so far? |
| `git diff` | What exactly did I change? |
| `git restore <file>` | Undo my edits in one file |

> **Remember**
> Git is not scary. **status** never changes anything, so run it as often as you like. Commit small and commit often.